Verilog HDL 考前 7 日练习

浙江大学 · 硬件描述语言（罗小华）

按课件第 02–08 章整理 · 每日概念要点 + 3 道编程题

考前冲刺用 · 关键词遗忘可用注释说明

目录

第 1 天　模块结构、数据类型、运算符与连续赋值 **2**

第 2 天　always、块语句、阻塞/非阻塞、循环与 generate **3**

第 3 天　建模方式与组合逻辑模块 **4**

第 4 天　时序逻辑：触发器、计数器、移位与复位 **5**

第 5 天　有限状态机 FSM **6**

第 6 天　可综合描述风格与常见陷阱 **7**

第 7 天　TestBench / 验证 + 综合复习编程 **8**

使用说明

- 你已通读过一遍：本册以**默写练习**为主，不要再从头翻书。
- 每天：先读「概念要点」，再独立完成 3 道编程题（可综合风格优先）。
- 考试规则对齐：个别关键字忘了，可在代码旁用注释写清含义。
- 每道题写完后自查：端口类型、`wire`/`reg`、阻塞/非阻塞、敏感列表、意外锁存。

第 1 天　模块结构、数据类型、运算符与连续赋值

当日概念要点（对应课件第 02 章）

- **模块四部分**：端口表 → I/O 说明（input/output/inout）→ 信号类型（wire/reg/...）→ 功能描述。
- **三种功能描述**：assign 连续赋值；门/模块例化；always 过程块。assign 与 always 并发，assign 不能写在 always 内。
- **基本类型**：wire（网线，由驱动赋值）、reg（过程赋值变量，不一定是触发器）、parameter、integer。
- **常量**：位宽' 进制值，如 8'hA5、4'b1010；x/z；parameter 符号常量。
- **运算符**：算术、逻辑（&& || !）、按位（& | ^ ~）、归约、移位、条件 ?:、拼接 {} / 复制 {n{...}}。
- **例化**：位置关联与名字关联 .port(sig)；实例名应唯一。

编程题 1.1　写模块 alu_simple，实现 4 位简易 ALU。
端口：input [3:0] a, b; input [1:0] op; output [3:0] y;
功能：op=00 加法；01 按位与；10 按位或；11 按位异或。优先用 assign + 条件/case 等价写法（连续赋值侧可用条件运算符）。要求可综合。

编程题 1.2　写模块 bus_mux4：4 选 1 的 8 位总线多路选择器。
端口：input [7:0] d0,d1,d2,d3; input [1:0] sel; output [7:0] y;
分别给出（a）纯 assign 版本；（b）门级思路说明（不必完整门网表）：指出会用到哪些多路选择思想。主交付以（a）为准。

编程题 1.3　写模块 parity8：对 8 位输入 data 计算偶校验位 p（使 {p,data} 中 1 的个数为偶数）。
端口：input [7:0] data; output p;
提示：可用归约异或。再扩展一个 parameter WIDTH=8 的参数化版本接口说明（代码中写出参数即可）。

第 2 天 always、块语句、阻塞/非阻塞、循环与 generate

当日概念要点（对应课件第 03 章-2/3/4）

- **always**：被赋值对象须为 `reg`（等 register 类）。敏感表列出所有影响取值的信号；组合用电平敏感，时序用 `posedge/negedge`。
- **无时序控制的 always** 易仿真死锁（如 `always areg=~areg;`）。
- **同一变量**不能在多个 `always` 中赋值。
- **begin-end**：顺序块；**fork-join**：并行块（多用于 TB，通常不可综合）。
- **阻塞 = / 非阻塞 <=**：组合逻辑推荐阻塞；时序（时钟沿）推荐非阻塞，避免仿真与综合不一致。
- **循环**：`for`（可综合场景多，如位遍历）；`repeat/while/forever` 多数不可综合或仅限 TB。
- **generate**：用 `genvar` + `generate for/if` 复制硬件结构（如 N 个全加器位）。

编程题 2.1 用 `always`（组合敏感）写 7 人表决器 `vote7`（课件例题思想）：
`input [6:0] vote; output pass;`
赞成票 ≥ 4 则 `pass=1`。要求用 `for` 累计，注意 `sum`、`pass` 类型与敏感列表完整。

编程题 2.2 写同步模块 `shift_load8`：
`input clk, rst_n, load, sin; input [7:0] din; output reg [7:0] q;`
异步低有效复位清零；`load=1` 时并行装载 `din`；否则串行左移，`sin` 进入最低位。时钟上升沿，**非阻塞赋值**。

编程题 2.3 用 `generate` 写参数化总线按位与 `and_bus`：
`parameter N=8; input [N-1:0] a,b; output [N-1:0] y;`
`generate for` 对每一位例化（或位运算赋值）实现 `y[i]=a[i]&b[i]`。给出完整可综合代码。

第 3 天　建模方式与组合逻辑模块

当日概念要点（对应课件第 04 章）

- **结构化**：例化基本门（and/nand/or/nor/xor/xnor/buf/not/bufif*/notif*）或子模块。
- **数据流**：assign 连续赋值描述组合。
- **行为级**：always/initial 描述算法行为；RTL 综合常用行为级中的可综合子集。
- 门例化端口顺序：多输入门为（输出，输入...）；三态为（输出，输入，控制）。
- 组合逻辑要写全分支，否则可能推断**锁存器**。

编程题 3.1　用**结构化门级**描述 1 位全加器 fa1：
input a,b,cin; output sum, cout;
允许只使用 xor/and/or 等基本门例化（写出实例名）。

编程题 3.2　用行为级写 4-16 译码器 dec4to16：
input [3:0] a; input en; output reg [15:0] y;
en=0 时输出全 0；en=1 时独热译码。用 case 或移位写法，避免锁存。

编程题 3.3　写比较器 cmp8：
input [7:0] a,b; output eq, gt, lt;
三者互斥完备（$a{=}b/a{>}b/a{<}b$）。数据流或行为级均可，注明选用风格。

第 4 天　时序逻辑：触发器、计数器、移位与复位

当日概念要点

- 时序模板：`always @(posedge clk or negedge rst_n)` + 非阻塞。
- **同步复位**只放在时钟沿敏感表的 clk 上判断；**异步复位**复位信号进敏感表。
- 计数器：加法/减法、装载、使能、模 N（到 $N-1$ 归零）。
- 移位寄存器：逻辑移/循环移、串入并出、并入串出。
- 避免在同一 `always` 混用阻塞与非阻塞（课件可综合风格强调）。

编程题 4.1　写模 10 计数器 `cnt_mod10`：
`input clk, rst_n, en; output reg [3:0] q; output cout;`
异步复位；`en=1` 时每时钟 +1，到 9 下一拍回 0 且该拍 `cout=1`（进位脉冲约定：当 `q==9 & en` 时可组合产生 `cout`）。说明你的 `cout` 是组合还是寄存器输出。

编程题 4.2　写可逆计数器 `updown4`：
`input clk, rst_n, up; output reg [3:0] q;`
同步复位（高有效 `rst` 也可，题目用 `rst_n` 异步）；`up=1` 加，`up=0` 减；自由回绕。

编程题 4.3　写并入串出移位寄存器 `piso8`：
`input clk, rst_n, load, shift; input [7:0] din; output sout;`
`load` 优先装载；否则 `shift=1` 时左移并输出原最高位到 `sout`；`load=shift=0` 保持。

第 5 天　有限状态机 FSM

当日概念要点（对应课件第 05 章）

- **Moore**：输出只依赖状态；**Mealy**：输出依赖状态 + 输入。
- **一段式 / 二段式 / 三段式**：推荐二段或三段——状态寄存器时序 + 次态组合（+ 输出单独一段）。
- **编码**：二进制、格雷、独热（one-hot）；FPGA 常偏独热，ASIC 视情况。
- **安全状态机**：`default` 回到空闲/复位态，防止非法状态死锁。
- 设计步骤：状态图 → 编码 → 状态/输出方程 → Verilog。

编程题 5.1　设计 Moore 序列检测器 `seq_1101_moore`：检测输入串中 “1101”（可重叠）。
`input clk, rst_n, din; output reg found;`
用二段或三段式；画出你采用的状态名（注释即可）；非法/默认回 `S0`。

编程题 5.2　设计 Mealy 版本 `seq_1101_mealy`：同样检测 “1101”，输出可为组合（随 `din` 立即变）。对比说明与 5.1 输出时序差异（3–5 句注释写在模块头）。

编程题 5.3　简易交通灯控制 `traffic_light`（简化）：
两灯 `ns_light[1:0]`, `ew_light[1:0]`，编码：`00` 红 `01` 黄 `10` 绿。
状态：NS 绿 –tick–> NS 黄 –tick–> EW 绿 –tick–> EW 黄 –tick–> 循环。
`input clk, rst_n, tick;`（`tick` 为计时到脉冲）。用 FSM 实现；复位后 NS 绿、EW 红。

第 6 天　可综合描述风格与常见陷阱

当日概念要点（对应课件第 06 章-1，兼第 07 章认知）

- **通常不可综合**：`initial`、`forever/while/repeat`（多数）、`fork-join`、`wait`、`force/release`、`===/!==`、`real/time/event`、UDP 等。
- **敏感列表**不全 ⇒ 仿真与综合不一致；组合推荐 `@*` / `@(*)`。
- **锁存推断**：组合 `if/case` 未覆盖所有条件/分支。
- **阻塞 vs 非阻塞**：时序非阻塞、组合阻塞；不要在同一 always 混用。
- **复位**：异步复位注意释放异步、采样同步的工程问题（了解即可）。
- **延时模型（第 07 章）**：块延时 / 分布延时 / `specify` 路径延时——**综合会忽略 `#` 延时**；考试简答常考「仿真有延时、综合无」。

本日穿插简答自测（不必交，口头答）：
(1) 为何组合逻辑漏写 `else` 会出锁存?
(2) 列出至少 5 种常见不可综合结构。
(3) `specify` 路径延时与门上 `#` 分布延时差别一句话。

编程题 6.1　改错题：指出下列风格问题并给出可综合改写（保留功能：时钟上升沿采样 `d` 到 `q`，异步低复位）。

```
module dff_bad(input clk, rst_n, d, output q);
  reg q;
  always @(clk or rst_n) begin
    if (!rst_n) q = 0;
    else if (clk) q = d;  // intend posedge
  end
endmodule
```

编程题 6.2　写优先编码器 `prienc8`：
`input [7:0] in; output reg [2:0] code; output reg valid;`
最高优先级为 `in[7]`。用 `casex/case` 或 `if-else` 链；保证无锁存。

编程题 6.3　写带同步使能的流水线寄存器级 `pipe_reg`：
`parameter W=16; input clk, rst_n, valid_i; input [W-1:0] data_i;`
`output reg valid_o; output reg [W-1:0] data_o;`
`valid_i` 为 1 时本拍装载数据并传递 valid；为 0 时 data 保持、valid 清 0（或保持——题目要求：**valid 清 0，data 保持**）。注意非阻塞与复位。

第 7 天　TestBench / 验证 + 综合复习编程

当日概念要点（对应课件第 08 章）

- TB 常用 `initial` 产生激励；`always #half clk=~clk` 产生时钟。
- 显示：`$display`/`$write`/`$strobe`/`$monitor`；时间：`$time`/`$stime`/`$realtime`，`$timeformat`。
- 文件 I/O（了解）：`$fopen`/`$fdisplay`/`$fread` 等。
- `$finish`/`$stop` 结束仿真。
- TB 本身通常**不综合**；DUT 保持可综合。

编程题 7.1　为第 4 天的 `cnt_mod10` 写 testbench `tb_cnt_mod10`：
产生 10ns 周期时钟；复位释放后使能计数至少 25 个时钟；用 `$monitor` 打印 `time,q,cout`；结束调用 `$finish`。不必接波形文件。

编程题 7.2　综合题：写同步 FIFO 简化版（深度 4，宽度 8）`fifo4x8`：
`input clk, rst_n, wr_en, rd_en; input [7:0] din;`
`output reg [7:0] dout; output full, empty;`
指针或计数实现；同时读写行为自行约定并注释。重点：满/空标志正确、复位。

编程题 7.3　综合题：写握手数据通道 `ready_valid_reg`：
`input clk, rst_n; input valid_i, ready_i; input [7:0] data_i;`
`output reg valid_o; output ready_o; output reg [7:0] data_o;`
单级寄存器切片：当下游 ready 或本级无有效数据时可接收上游；给出 `ready_o` 组合逻辑表达式并实现。

附录：考前速查（半页）

- 组合：`assign` 或 `always @*` + 阻塞 `=`；分支写全防锁存。
- 时序：`always @(posedge clk ...)` + 非阻塞 `<=`。
- FSM：二/三段式 + `default` 安全态；先画状态图再写码。
- wire/reg：过程赋值 → reg；连续赋值/端口连接 → wire。
- 忘关键字：注释说明意图，把逻辑写对。

对应课件：第 *02–08* 章（第 *00/01* 章工具与绪论可略；*Project* 专题不纳入本练习）

PDF 与 *main.tex* 同源，可按需增删题目。